



# Sistemas Distribuidos I (75.74)

## MOM Distribuido: ZeroMQ

Ejemplos Prácticos.

### Docentes

- Pablo D. Roca
- Gabriel Robles
- Franco Barreneche
- Tomás Nocetti
- Nicolás Zulaica

# Agenda

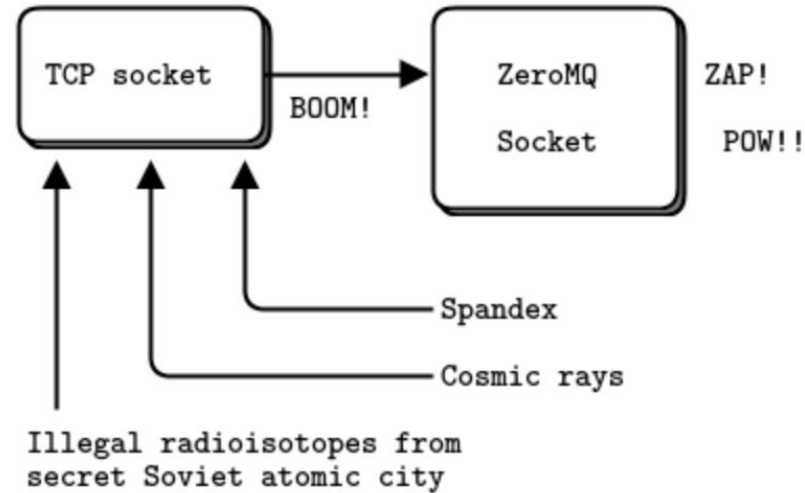


## ● ZeroMQ

# ZeroMQ | Definición según sus autores



We took a normal TCP socket, injected it with a mix of radioactive isotopes stolen from a secret Soviet atomic research project, bombarded it with 1950-era cosmic rays, and put it into the hands of a drug-addled comic book author with a badly-disguised fetish for bulging muscles clad in spandex. Yes, ZeroMQ sockets are the world-saving superheroes of the networking world.





# ZeroMQ | Introducción

- Sockets on steroids
- Altamente performante, aunque hay alternativas más modernas:
  - Nanomsg
  - NSQ
- Herramienta útil para crear **brokerless middlewares**
- Serialización **a cargo** del usuario
- Soporte para diferentes patrones de mensajería
  - Request-Reply, Publisher-Subscriber, Parallel Pipeline, Patrones avanzados





# ZeroMQ | Tipos de conexiones

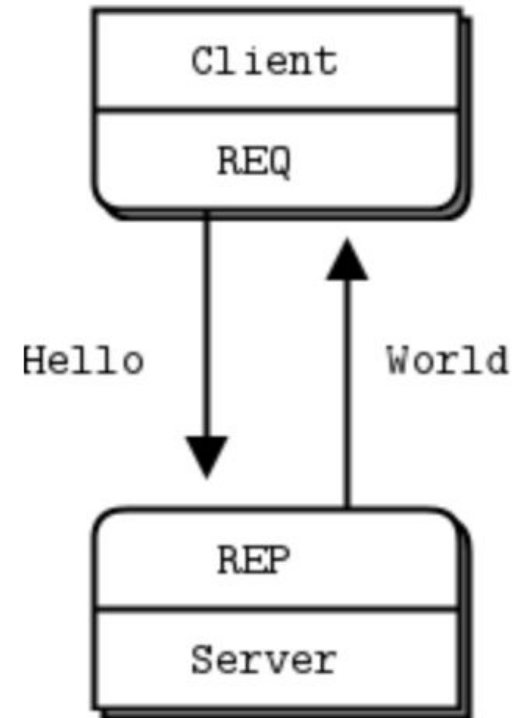
- **TCP**
  - Multicomputing
  - Unicast (Point to Point)
- **IPC**
  - Multiprocessing
  - Comunicación a través de Unix sockets
- **Inproc**
  - Multithreading
  - Queue entre threads
- **Otras**
  - Multicast a través del protocolo [PGM](#)





# ZeroMQ | Patrones | Request-Reply

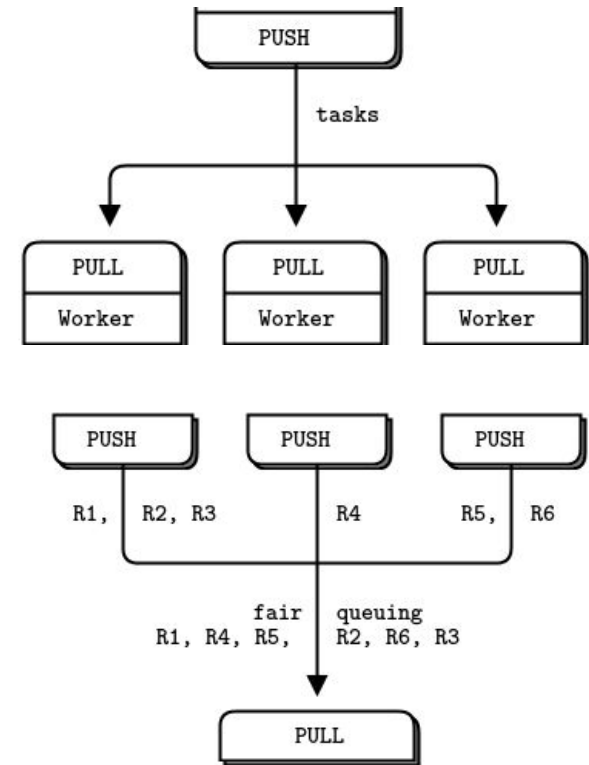
- Cliente-Servidor convencional (aunque no tanto...)
- No posee primitiva *accept*
- Primitiva *bind* funciona como *bind + accept*
- Primitiva send es **no bloqueante**
- Cliente no necesita esperar a que el servidor esté corriendo para enviar mensajes
- Cómo funciona *under the hood*?
  - I/O threads: 1 thread per GB/s (in or out)
  - Buffering





# ZeroMQ | Patrones | Producer-Consumer (o Push-Pull)

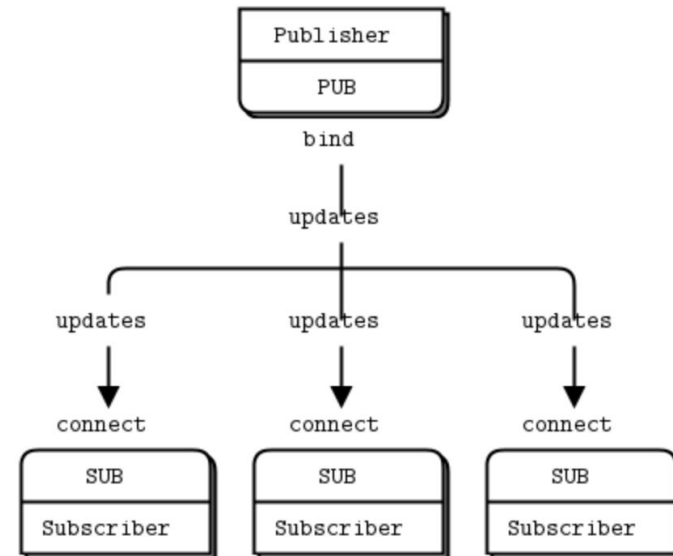
- Comunicación de tareas de un productor a un consumidor.
- Admite múltiples consumidores y/o múltiples productores.
- Garantiza *fairness* en la entrega de mensajes: round robin
- Utiliza los sockets PUSH/PULL para marcar el rol de cada extremo.





# ZeroMQ | Patrones | Publisher-Subscriber

- Un ZMQ PUB socket publica mensajes
- Message pattern: `id field1 field2 ...fieldN`
- N ZMQ SUB sockets se registran a los Eventos que desean recibir suscribiendose al ID del evento
- Suscripción puede ser cancelada en cualquier momento
- Mensaje es enviado a todos los sockets suscriptos a un evento determinado
- Múltiples publishers? [XPUB-XSUB pattern](#)

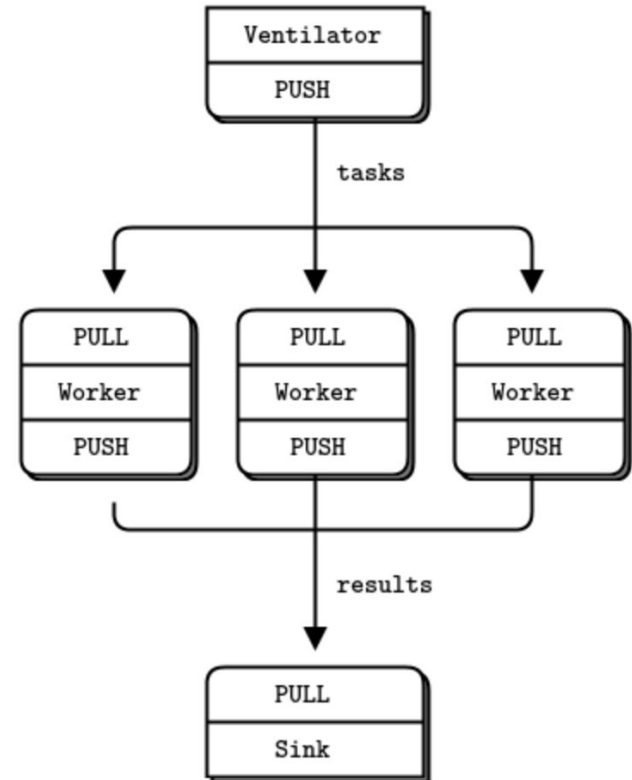






# ZeroMQ | Patrones | Pipeline (Push - Pull)

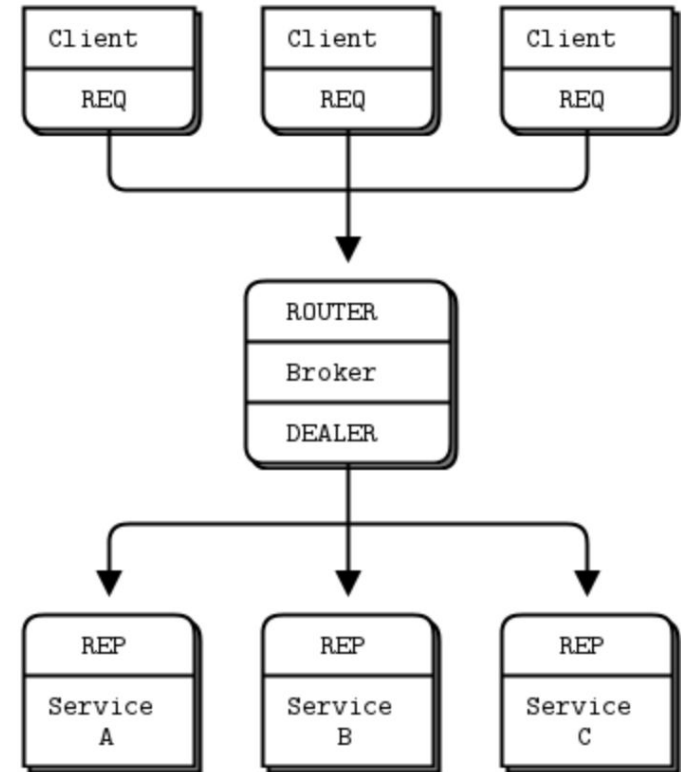
- Patrón Productor-Consumidor
- Chaining de Productores-Consumidores da como resultado un pipeline
- Mensajes son consumidos de forma Equitativa (**fairness**)
  - Qué lógica utiliza para decidir esto?
- Combinaciones
  - 1 PUSH -> N PULL
  - N PUSH -> 1 PULL





# ZeroMQ | Patrones | Router-Dealer (Broker)

- **ROUTER socket**
  - Agrega al mensaje recibido un ID de destinatario
- **DEALER socket**
  - Rutea los mensajes de forma justa (**fair**)
  - Propaga el ID de origen del mensaje
- Ambos sockets permiten recibir mensajes de múltiples sockets a la vez
- Ambos sockets son asíncronos
  - **Poll** para recibir mensajes





# Bibliografía

- ZeroMQ: The Guide: <http://zguide.zeromq.org/page:all>
- Ejemplos ZeroMQ:  
<https://github.com/7574-sistemas-distribuidos/zeromq-examples>